-Student Name: Danelle Nicole G. Jumento

-Group:

-Assigned Role: Developer / Programmer

-What did I contribute?

Contributed to the core development of the project by implementing essential process management modules, including functionalities to initialize, monitor, execute, and safely terminate system processes.

-What features did you personally develop?

* Process Execution ("Run"): Developed the logic and program code required to successfully launch and run target tasks or processes.

* Process Management: Built monitoring mechanisms to track and manage active processes within the system.

* Process Termination ("Kill"): Implemented safe and reliable functions to halt or terminate running processes when necessary.

-What problem did you encounter?

Encountered various debugging challenges, including persistent runtime errors, unhandled exceptions, and instances where the code failed to execute as intended during initial testing phases.

-How did you solve those problems?

Overcame technical hurdles by conducting targeted research through online developer resources, technical documentation, and community forums to diagnose root causes and implement effective bug fixes.

-What did you learn from this project?

Gained valuable hands-on problem-solving skills, improved debugging and troubleshooting techniques, and developed greater resilience when tackling complex technical challenges.

-Evidence of my contribution:

```
# ------------------------------------------------------------------------

# Process Management (process, run, kill)

# ------------------------------------------------------------------------

class ProcessManager:

    def __init__(self):

        self.processes = [
```

```
        {"pid": 1, "name": "system_idle", "status": "RUNNING"},
        {"pid": 2, "name": "shell", "status": "RUNNING"}
    ]
    self.next_pid = 3

def list_processes(self):
    lines = [f"\n{'PID':<6} {'NAME':<15} {'STATUS':<10}", "-" * 33]
    for p in self.processes:
        lines.append(f"{p['pid']:<6} {p['name']:<15} {p['status']:<10}")
    return "\n".join(lines)

def run_process(self, name):
    if not name.strip():
        raise FSError("Process name cannot be empty.")
    new_proc = {
        "pid": self.next_pid,
        "name": name.strip(),
        "status": "RUNNING"
    }
    self.processes.append(new_proc)
          output = (
        "Process started.\n\n"
        f"PID: {new_proc['pid']}\n"
        f"Name: {new_proc['name']}\n"
        f"Status: {new_proc['status']}"
    )
    self.next_pid += 1
    return output

def kill_process(self, pid_str):
```

```
try:
    pid = int(pid_str)
except ValueError:
    raise FSError("Invalid PID format. Please provide a numeric PID.")

for p in self.processes:
    if p["pid"] == pid:
            if p["name"] in ["system_idle", "shell"] and pid <= 2:
            raise FSError(f"Cannot terminate core system process '{p['name']}'.")
            self.processes.remove(p)
            return f"Process {pid} terminated."
raise FSError("Process does not exist.")
```